

Scaling a robot dispatch API: concurrency inside one service, then distribution across replicas

1. Introduction and scope

This project takes the robot dispatch path of my SIT331 backend (an ASP.NET Core 8 API over PostgreSQL) and makes it faster and able to scale out. Robots run a loop of three HTTP requests per job:

```
POST /api/adapters/devices/{id}/work-items/claim-next -> 200 job | 204 no work | 429 come back later
PATCH /api/adapters/jobs/{jobId}/started
PATCH /api/adapters/jobs/{jobId}/completed
```

The SIT331 version works for a handful of robots. Under load it slows down, then stops completing jobs at all. The SIT315 work is in two layers:

- **Concurrency inside one API:** remove the database bottlenecks on the dispatch path, make the path asynchronous, make the job claim atomic, and add backpressure (a bounded concurrency limiter that answers 429 with Retry-After).
- **Distribution across APIs:** run several stateless replicas behind a load balancer against one shared PostgreSQL, with the background sweep coordinated through a PostgreSQL advisory lock so only one replica runs it at a time.

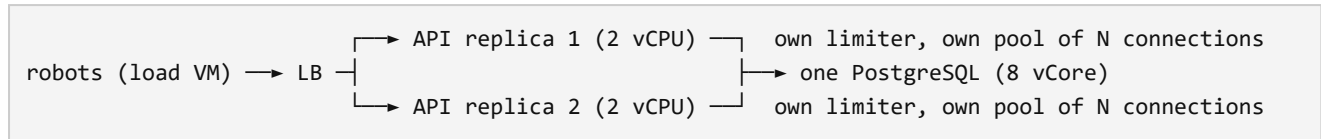
The unit taught threads, OpenMP, MPI and OpenCL on compute-bound problems. This project applies the same ideas (contention, task decomposition, bounded buffers, scaling out) to an I/O-bound request service, using techniques the unit did not cover: asynchronous I/O on a thread pool, admission control, connection pooling, prepared statements, optimistic atomic updates and lock-based coordination between replicas. Research for it covered the ASP.NET Core rate limiting middleware, Npgsql connection pooling and automatic statement preparation, PostgreSQL advisory locks and row level concurrency, and closed loop load testing with NBomber. The project was approved by the unit chair on the basis that the SIT331 part and the SIT315 part are clearly separated.

Where the code is. The repository has several branches:

Branch	What it holds
new	the main branch: the final SIT315 code. Read this one to see the solution
old	the SIT331 code exactly as benchmarked (the SIT331 version plus one commit that only moves ignore entries)
phase-1 to phase-4	the step by step additions, each phase built on the one before; phase-4 was merged into new
loadgen	the load generator, added between phase 3 and phase 4

The tag `sit331-baseline` marks the last SIT331 commit. `git diff --stat old..new` shows the SIT315 work: 71 files changed, 3,037 lines added and 497 removed, of which 1,852 added lines are the load generator and the new tests. In this report "old" means the `old` branch and "new" means the `new` branch.

2. System and test environment



All measurements were taken on Microsoft Azure, region Korea Central, on 2026-10-04.

Part	Specification
API VM 1	Standard_D2s_v4, 2 vCPU, 7.8 GB, Intel Xeon Platinum 8370C @ 2.80 GHz, Docker
API VM 2	Standard_D2s_v4, 2 vCPU, 7.8 GB, Intel Xeon Platinum 8272CL @ 2.60 GHz, Docker
Load VM	Standard_D2ds_v4, 2 vCPU, 7.8 GB, Intel Xeon Platinum 8272CL @ 2.60 GHz
Database	Azure Database for PostgreSQL Flexible Server 18, Standard_D8ds_v5, 8 vCore, 128 GB, private subnet, TLS off
Load balancer	Azure Standard internal load balancer, port 80 to 8080, HTTP health probe on /health every 5 s
Connection pool	Npgsql Maximum Pool Size 25 per replica (main grid), 50 per replica (second grid)
Images	robot-api:old = old branch (9faf8b1), robot-api:new = new branch (a87f92d), load generator = new branch

Development and the unit tests ran on my laptop: AMD Ryzen 5 7640HS, 6 cores / 12 threads, 16 GB RAM, Windows 11. No laptop numbers are used in this report.

3. Baseline analysis: what limited the old code

I read the dispatch path end to end and listed every cost or race on it. The F numbers are used through the rest of the report.

#	Problem in the old code	Kind
F1	claim was a read then a separate write, so two robots could claim the same job	correctness
F2	started, completed and failed overwrote the job without checking its state, so updates could be lost	correctness
F3	authentication loaded every device credential from the database, then searched the list in C#	DB trips + API CPU
F4	authentication wrote LastUsed metadata back to the database on every request	extra DB write per request
F5	every claim-next scanned the robot's whole job history for stale work	extra DB work per claim
F6	the stale sweep loaded every job, and every replica would run its own sweep	DB work, multi-replica conflict
F7	all database I/O on the dispatch path was synchronous	threads blocked during every query
F8	no limit on how many requests are admitted	no backpressure, collapse under overload
F9	a second ownership query repeated a check already done	extra DB trip
F10	an unused StopWithoutWork decision	dead code
F11	the claim query read all of a robot's queued jobs to keep the oldest one	expensive DB query
F12	every SQL statement was parsed and planned again on every execution	DB CPU per query

Grouped by effect, there are five bottlenecks: too many database trips per request (F3, F4, F5, F9), each trip too expensive (F3, F11, F12), threads asleep during database waits (F7), no admission control (F8), and the

fixed capacity of one API process (2 vCPU, one pool), which only scaling out removes.

4. Design and implementation

The work was added in four small phases on stacked branches (phase-1 to phase-4), each reviewed and tested before the next. The last phase was merged into `new`, so `new` contains all four.

4.1 Phase 1: correctness and fewer queries (F1 to F5, F9, F10)

The claim became one guarded statement. Only one concurrent caller can match `status = 'Queued'`, every other caller gets zero rows back and moves on to the next job, so two robots can never hold the same job:

```
UPDATE public.job
SET status = 'Claimed', claimedbydevicecredentialid = @deviceCredentialId, claimedatutc =
@claimedAtUtc, ...
WHERE id = @id AND status = 'Queued'
RETURNING ...;
```

started, completed and failed use the same guard on the expected current state and the robot's `claimedAtUtc`, so a stale or duplicate update changes nothing (F2). Authentication now fetches one credential row by its identifier (F3) and writes `LastUsed` at most once every 10 minutes (F4). The duplicate ownership query was removed (F9).

4.2 Phase 2: background stale sweep coordinated across replicas (F5, F6)

Stale work is no longer checked inside `claim-next`. A background service sweeps every 15 s with one targeted query (F6a). With several replicas, each one runs the service, so the sweep is wrapped in a transaction scoped PostgreSQL advisory lock (F6b). The replica that gets `pg_try_advisory_xact_lock(315331001)` sweeps; the others skip that tick. The lock is released automatically when the transaction ends, so a crashed replica cannot hold it.

4.3 Phase 3: asynchronous dispatch path and backpressure (F7, F8)

Every database call on the four robot endpoints became `async/await`. The thread returns to the pool while PostgreSQL works and any free thread continues the request when the answer arrives:

```
sync (old): thread T1: [parse][== blocked on PostgreSQL ==][reply]
async (new): thread T1: [parse] serves other requests [reply] (may be a different thread)
```

The endpoints share one ASP.NET Core concurrency limiter (`Infrastructure/DispatchBackpressure.cs`):

```
limiter.PermitLimit = permitLimit; //requests allowed to run at the same time
limiter.QueueLimit = queueLimit; //requests allowed to wait for a permit
limiter.QueueProcessingOrder = QueueProcessingOrder.OldestFirst; //waiting robots are served in
arrival order
options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
```

With the defaults, 64 requests run per replica, 32 wait in arrival order, and the rest are answered at once with 429 and `Retry-After: 1`. This is a bounded buffer with a signal back to the sender, which is backpressure: the robot slows down instead of the server buffering without limit. The limiter runs before

authentication, so a rejected request costs no database work. It caps work in progress rather than requests per second, so it adapts to how fast the database is.

4.4 Phase 4: cheaper queries (F11, F12)

The claim query now reads the oldest queued job through an index with `LIMIT 1` instead of reading every queued job of the robot. Prepared statements are switched on through `Npgsql (Max Auto Prepare = 64` in `DbConfig.cs`), so a statement that repeats on a connection is parsed and planned once and afterwards only executed.

4.5 Distribution

The API holds no state between requests, so replicas need no coordination apart from the sweep lock in 4.2. Each replica has its own limiter and its own connection pool; the load balancer spreads robots across healthy replicas; all replicas share one PostgreSQL. Replica count is switched by starting or stopping the container on the second VM and letting the health probe take it in or out.

5. Correctness

- **Tests:** 54 automated tests pass on `new` (integration tests that need a running API excluded), run against a real PostgreSQL in Docker (Testcontainers). They include concurrent claim tests (many robots claiming at once, every job claimed exactly once), guarded transition tests, the advisory lock, and the limiter (queueing, then 429 with Retry-After).
- **After every benchmark run** the load generator checks the database. Table 1 adds those checks up over both grids.

Table 1. Database check after every grid run (both pool sizes).

pool	runs	jobs completed (DB)	history rows	match	failed	other status	claimed or executing at stop
25	36	565,394	565,394	yes	0	0	11 in 1 run(s)
50	36	641,739	641,739	yes	0	0	0 in 0 run(s)

Every completed job has exactly one history row, no job ended Failed and no job ended in an unknown state. "Claimed or executing at stop" counts jobs that were still in the middle of their 3 requests when the 30 s window closed. The only run with any was an old, 1 API, 400 robot repeat with pool 25, which also logged 12 failed requests.

6. Test method

- **Load:** NBomber, closed loop. Each simulated robot sends its next request the moment the previous answer arrives, so the number of robots is the number of requests in flight. On 429 it waits Retry-After and tries again; started and completed are retried until they land (up to 20 times) so a job is never left half done.
- **Workload:** every robot has a backlog of 2,000 queued jobs, re-seeded before each run.
- **Sizes:** 10, 100 and 400 robots. Stress: 2,000 and 5,000 robots.
- **Setups:** old and new, each with 1 and 2 API replicas, at pool size 25 and again at pool size 50.
- **Runs:** 30 s measured after a 5 s warm-up, 3 repeats per point, median reported with the min to max range. 72 grid runs and 8 stress runs in total.

Metrics:

Metric	Meaning
jobs/s	completed jobs per second (completed requests that succeeded)
req/s	successful requests per second over all three steps (about 3 x jobs/s)
time per job	robots / jobs/s (Little's law for a closed loop). How long one robot takes per job, including any 429 waits
p50 / p95 / p99	latency of successful requests only. A 429 counts as a failed request and the Retry-After wait is outside the timing
speedup	jobs/s of a setup / jobs/s of old, 1 API at the same size and pool. Equal to the reduction in time per job

Because latency only covers requests that got in, it is always read together with the 429 count and jobs/s.

7. Results (pool size 25)

7.1 Throughput

Table 2. Throughput, pool 25 (median of 3).

robots	setup	jobs/s median (min to max)	req/s	time per job (ms)	429s	other fails
10	Old, 1 API	179.7 (172.5 to 180.7)	539	56	0	0
10	Old, 2 APIs	168.7 (152.2 to 176.2)	506	59	0	0
10	New, 1 API	341.5 (338.3 to 342.4)	1,024	29	0	0
10	New, 2 APIs	332.2 (328.2 to 334.9)	997	30	0	0
100	Old, 1 API	246.1 (226.0 to 250.4)	738	406	0	0
100	Old, 2 APIs	363.9 (355.0 to 375.9)	1,092	275	0	0
100	New, 1 API	617.9 (616.0 to 618.5)	1,854	162	120	0
100	New, 2 APIs	1,102.9 (1,077.9 to 1,103.2)	3,309	91	0	0
400	Old, 1 API	176.7 (152.8 to 192.4)	530	2,263	0	0
400	Old, 2 APIs	284.2 (231.6 to 290.2)	853	1,407	0	0
400	New, 1 API	607.7 (602.2 to 608.4)	1,823	658	9,120	0
400	New, 2 APIs	1,069.0 (1,067.7 to 1,082.2)	3,207	374	6,337	0

7.2 Speedup over the baseline (old, 1 API)

Table 3. Speedup over old, 1 API, pool 25.

robots	setup vs Old, 1 API	jobs/s speedup	time per job ms (old 1 -> setup)	time per job speedup
10	Old, 2 APIs	0.94x	56 -> 59	0.94x
10	New, 1 API	1.90x	56 -> 29	1.90x
10	New, 2 APIs	1.85x	56 -> 30	1.85x
100	Old, 2 APIs	1.48x	406 -> 275	1.48x
100	New, 1 API	2.51x	406 -> 162	2.51x
100	New, 2 APIs	4.48x	406 -> 91	4.48x
400	Old, 2 APIs	1.61x	2,263 -> 1,407	1.61x
400	New, 1 API	3.44x	2,263 -> 658	3.44x
400	New, 2 APIs	6.05x	2,263 -> 374	6.05x

The headline at 400 robots: old with 2 APIs reaches 1.61x, new with 1 API 3.44x, and new with 2 APIs 6.05x. Time per job drops from 2,263 ms to 374 ms.

7.3 Replica scaling

Table 4. Effect of the second API replica, both pool sizes.

pool	robots	version	1 API jobs/s	2 APIs jobs/s	2 vs 1 API
25	10	old	179.7	168.7	0.94x
25	10	new	341.5	332.2	0.97x
25	100	old	246.1	363.9	1.48x
25	100	new	617.9	1,102.9	1.78x
25	400	old	176.7	284.2	1.61x
25	400	new	607.7	1,069.0	1.76x
50	10	old	175.0	183.2	1.05x
50	10	new	325.0	347.2	1.07x
50	100	old	236.5	387.0	1.64x
50	100	new	752.8	1,276.2	1.70x
50	400	old	188.9	311.8	1.65x
50	400	new	736.8	1,240.3	1.68x

7.4 Latency

Table 5. Latency of successful requests, pool 25 (median of the 3 runs' percentiles).

robots	setup	claim-next p50 / p95 / p99 ms	started p95 ms	completed p95 ms
10	Old, 1 API	24.0 / 30.5 / 35.2	14.0	24.8
10	Old, 2 APIs	25.7 / 32.1 / 36.3	15.1	27.1
10	New, 1 API	9.7 / 12.0 / 14.4	7.1	16.2
10	New, 2 APIs	9.9 / 12.2 / 15.1	7.3	16.5
100	Old, 1 API	132.5 / 217.7 / 267.3	196.0	211.1
100	Old, 2 APIs	86.3 / 171.3 / 225.3	147.7	163.2
100	New, 1 API	58.9 / 66.9 / 86.8	40.1	67.6
100	New, 2 APIs	35.0 / 44.8 / 60.8	21.0	48.4
400	Old, 1 API	766.5 / 1,330.2 / 1,603.6	1,137.7	1,158.1
400	Old, 2 APIs	453.6 / 939.0 / 1,332.2	742.9	785.9
400	New, 1 API	59.5 / 67.1 / 84.9	40.5	68.0
400	New, 2 APIs	65.4 / 79.9 / 93.4	47.8	81.3

Why speedup is measured on time per job, not on latency. Speedup in this unit means the same work done in less time. In a closed loop, jobs/s and time per job are the same measurement (time per job = robots / jobs/s), so the jobs/s speedup is already a time based speedup. Latency answers a narrower question and would mislead here:

- it only covers requests that got in, and the limiter keeps new's latency low partly by answering the rest with 429;
- it is one request, while a job is three requests plus any Retry-After waits;

- so it overstates the gain under load.

At 400 robots, pool 25, old against new with 1 API:

Measure	Old, 1 API	New, 1 API	Ratio
time per job (what a robot experiences)	2,263 ms	658 ms	3.44x
claim-next p50 (successful requests only)	766.5 ms	59.5 ms	12.9x

Robots finished jobs 3.44x faster, not 12.9x. Most of the difference is the Retry-After waiting after 429s, which p50 does not see; the rest is that a job is three requests, not one. Time per job is therefore the speedup used in this report, and latency is reported separately as responsiveness.

8. Results with pool size 50

The same grid was repeated with Maximum Pool Size 50 per replica.

Table 6. Throughput, pool 50 (median of 3).

robots	setup	jobs/s median (min to max)	req/s	time per job (ms)	429s	other fails
10	Old, 1 API	175.0 (168.5 to 177.1)	525	57	0	0
10	Old, 2 APIs	183.2 (176.4 to 196.9)	550	55	0	0
10	New, 1 API	325.0 (321.1 to 325.3)	975	31	0	0
10	New, 2 APIs	347.2 (330.0 to 350.5)	1,042	29	0	0
100	Old, 1 API	236.5 (232.2 to 261.2)	710	423	0	0
100	Old, 2 APIs	387.0 (374.4 to 409.4)	1,161	258	0	0
100	New, 1 API	752.8 (748.4 to 756.4)	2,258	133	113	0
100	New, 2 APIs	1,276.2 (1,268.8 to 1,285.9)	3,829	78	0	0
400	Old, 1 API	188.9 (144.9 to 209.2)	567	2,118	0	0
400	Old, 2 APIs	311.8 (309.4 to 345.5)	935	1,283	0	0
400	New, 1 API	736.8 (734.1 to 739.9)	2,210	543	9,099	0
400	New, 2 APIs	1,240.3 (1,236.8 to 1,260.3)	3,721	323	6,210	0

Table 7. Speedup over old, 1 API, pool 50.

robots	setup vs Old, 1 API	jobs/s speedup	time per job ms (old 1 -> setup)	time per job speedup
10	Old, 2 APIs	1.05x	57 -> 55	1.05x
10	New, 1 API	1.86x	57 -> 31	1.86x
10	New, 2 APIs	1.98x	57 -> 29	1.98x
100	Old, 2 APIs	1.64x	423 -> 258	1.64x
100	New, 1 API	3.18x	423 -> 133	3.18x
100	New, 2 APIs	5.40x	423 -> 78	5.40x
400	Old, 2 APIs	1.65x	2,118 -> 1,283	1.65x
400	New, 1 API	3.90x	2,118 -> 543	3.90x
400	New, 2 APIs	6.57x	2,118 -> 323	6.57x

Table 8. Pool 25 against pool 50.

robots	setup	pool 25 jobs/s	pool 50 jobs/s	change	claim p50 ms (25 -> 50)
10	Old, 1 API	179.7	175.0	-3%	24.0 -> 24.3
10	Old, 2 APIs	168.7	183.2	+9%	25.7 -> 23.7
10	New, 1 API	341.5	325.0	-5%	9.7 -> 10.2
10	New, 2 APIs	332.2	347.2	+5%	9.9 -> 9.6
100	Old, 1 API	246.1	236.5	-4%	132.5 -> 136.6
100	Old, 2 APIs	363.9	387.0	+6%	86.3 -> 86.0
100	New, 1 API	617.9	752.8	+22%	58.9 -> 47.2
100	New, 2 APIs	1,102.9	1,276.2	+16%	35.0 -> 25.9
400	Old, 1 API	176.7	188.9	+7%	766.5 -> 762.9
400	Old, 2 APIs	284.2	311.8	+10%	453.6 -> 419.1
400	New, 1 API	607.7	736.8	+21%	59.5 -> 48.5
400	New, 2 APIs	1,069.0	1,240.3	+16%	65.4 -> 55.5

9. Stress test that fails

Old with 1 API and new with 2 APIs were pushed to 2,000 and 5,000 robots (backlog 200 jobs each, one run per point, 600 s wall limit per setup). A setup fails if it stops completing jobs or leaves work stuck.

Table 9. Stress runs, both pool sizes.

pool	robots	setup	jobs/s	claim p95 ms	429s	other fails	of which 500s	DB completed	claimed or executing at stop
25	2000	Old, 1 API	65.2	33,079	0	44	42	3,920	37
25	2000	New, 2 APIs	989.7	142	54,298	0	0	32,404	32
25	5000	Old, 1 API	0.0	59,376	0	2,881	12	135	9,342
25	5000	New, 2 APIs	1,007.8	99	146,172	0	0	36,405	450
50	2000	Old, 1 API	70.4	15,647	0	0	0	4,113	0
50	2000	New, 2 APIs	1,093.2	145	54,212	0	0	35,575	10
50	5000	Old, 1 API	6.9	57,573	0	3,377	0	706	9,143
50	5000	New, 2 APIs	1,174.5	93	145,782	0	0	42,181	217

Old, 1 API fails. At 5,000 robots with pool 25 it completed 0.0 jobs/s in the measured window, 2,881 requests failed, the claim p95 was 59,376 ms, and 9,342 jobs were left claimed or executing. Pool 50 did not rescue it: 6.9 jobs/s and 3,377 failed requests. At 2,000 robots with pool 25 it also returned 42 HTTP 500 errors. The 600 s wall limit was never reached (old finished in 209 s and 186 s): it did not run out of time, it collapsed.

New with 2 APIs held 1,007.8 jobs/s at 5,000 robots with pool 25 and 1,174.5 with pool 50, with no failed requests apart from 429. It answered 146,172 requests with 429 in the 30 s window. The new setup's

breaking point was not reached, because one 2 vCPU load VM cannot drive more.

One irregularity: the pool 50, 5,000 robot old run shows 706 completed jobs but 744 history rows. Every grid run matched, so this only appeared in the collapsed old run; I have not traced its cause.

10. Discussion

10.1 Light load (10 robots): per-request work

At 10 robots nothing is queued, so the difference between old and new is the cost of one request. New is 1.90x faster with 1 API and the claim p50 falls from 24.0 ms to 9.7 ms. This comes from bottlenecks 1 and 2: fewer database trips per request (one credential row instead of the whole table, no LastUsed write, no stale scan, no duplicate check) and cheaper ones (index plus LIMIT 1, prepared statements). A second replica does nothing here (0.94x old, 0.97x new): one API is not busy, so there is nothing to share out.

10.2 Heavy load (100 and 400 robots): contention and backpressure

The speedup grows with load: 1.90x, 2.51x, 3.44x for new with 1 API. The old code does not just stop scaling, it gets worse. Old with 1 API drops from 246.1 jobs/s at 100 robots to 176.7 at 400, while its claim p50 rises from 24.0 to 132.5 to 766.5 ms. Two things cause that:

- **Threads asleep on the database (F7).** A synchronous call holds its thread through every query. With many robots every thread is blocked on PostgreSQL, new requests wait for a thread, and the thread pool only adds threads slowly.
- **No admission control (F8).** Every request is accepted, so more requests are in progress at once, more threads compete for the same 2 vCPU, and waiting grows without limit. Taken far enough this becomes the collapse in section 9, where robots time out while the server keeps working on requests nobody is waiting for.

New holds a flat claim p50 of 58.9 ms at 100 robots and 59.5 ms at 400. Async releases the thread during each query, and the limiter keeps at most 96 requests in each replica. Robots beyond that get a 429 and come back 1 s later (9,120 per run at 400 robots). That waiting is not in the latency, but it is in the time per job, which is still 3.44x shorter than old. New with 1 API delivers the same jobs/s at 100 and 400 robots (617.9 and 607.7): it is saturated by 100 robots and the limiter keeps it at that level instead of letting it degrade.

10.3 Scaling out

A second replica gives new 1.78x at 100 robots and 1.76x at 400, and old 1.48x and 1.61x. Each replica brings its own 2 vCPU, 64 permits and connection pool, so new, which is limited inside the API, gains most. It is below 2x because the database is shared and both replicas' connections land on the same 8 vCore server. Old gains less because its heavier queries put more load on that shared database per job. The second VM also has the slower CPU of the two (8272CL at 2.60 GHz against 8370C at 2.80 GHz), which may pull the two replica figures down slightly; I did not separate that effect.

10.4 Pool size: new was limited by connections, old was not

What changed. The pool is the set of open database connections each API keeps ready. A request borrows one for each query and gives it back straight after. The second grid only changed its size, from 25 to 50 per API.

What happened.

- New got faster: +22% (1 API) and +16% (2 APIs) at 100 robots, +21% and +16% at 400. Its claim p50 at 400 robots fell from 59.5 to 48.5 ms.
- Old did not clearly change. It moved between -4% and +10%, which is inside the spread of its own three repeats (Table 2 ranges).

Why. In new, up to 64 requests per API are allowed in at once, but with pool 25 only 25 of them can be talking to the database at the same moment. The rest wait for a connection. Because the code is async, that wait holds no thread, so the pool itself becomes the queue. A bigger pool means a shorter queue, so new gets faster.

```
new, pool 25: 64 requests in -> 25 at the database, 39 waiting for a connection
new, pool 50: 64 requests in -> 50 at the database, 14 waiting for a connection
```

Old never gets that far. Its requests are held up earlier, waiting for a thread and doing more work per request, so it most likely rarely has more than 25 requests at the database at once and the extra connections sit unused.

With pool 50 the best setup, new with 2 APIs, reaches 6.57x over old with 1 API at 400 robots, and 7.02x over the pool 25 baseline.

Where it stops paying. Every connection is a separate PostgreSQL process, and the database has 8 vCore. Two APIs with 50 connections each can put up to 100 queries on those 8 cores at once, and they then share the cores in turn. The data shows a first sign of this: with pool 50 the gain from a second API dropped from 1.76x to 1.68x at 400 robots. Database CPU was not recorded, so this is consistent with the shared database becoming the limit but does not prove it. Section 11 describes the test that would.

10.5 Which fix moved which number

Bottleneck	Fix	Visible in
too many DB trips	F3, F4, F5, F9	10 robot latency and the 1.90x at light load
trips too expensive	F11, F12	same, and lower DB load per job, which helps scaling out
threads asleep on DB	F7 async	old degrading from 100 to 400 robots while new stays flat
no admission control	F8 limiter	flat new latency with 429s, old collapse in the stress test
one API's capacity	2 replicas	1.76x new at 400 robots, nothing at 10
connection pool size	config	+16 to +22 % for new at 100 and 400 robots, nothing clear for old

10.6 Advanced work for the Level 3 tier

- **Hybrid parallelism:** asynchronous concurrency inside each process combined with distribution across machines.
- **Load balancing and backpressure:** load balancer across replicas, a bounded concurrency limiter with 429 and Retry-After, and a database lock so a background job runs on exactly one replica.
- **Test method:** cloud deployment with stated hardware, three workload sizes plus two stress sizes, three repeats with medians and ranges, two pool configurations, and a database conservation check after every run.

11. Limitations and future work

- **Retry-After of 1 s is probably too long.** New with 2 APIs handles about 3,200 requests per second, so each of the 64 permits is reused many times per second and a robot told to wait 1 s may sit idle while capacity is free. Only 1 s was tested. Next: 0.1, 0.25, 0.5 and 1 s at the stress size, possibly with random jitter.
- **The real robot firmware does not handle 429.** The benchmark robots honour Retry-After; the Nano firmware does not yet.
- **Latency excludes 429s**, as described in section 6.
- **Only 2 replicas were tested**, on two different CPU models, with one load VM. New's breaking point was not reached.
- **Pool size was only tested at 25 and 50.** More connections help until the database cores are full; past that, queries take turns on the same cores, switch between processes and wait on each other's locks, so throughput flattens and latency rises. A common starting point is about 2 to 3 busy connections per database core, so roughly 16 to 24 for this 8 vCore server, but the best value depends on the queries and has to be measured. Next: run new with 1 and 2 APIs at pool 10, 25, 50 and 100 per API, and record jobs/s, p95 and PostgreSQL CPU together. The best pool size is the point where jobs/s stops rising while p95 starts climbing. Section 10.4 suggests that point is close at 2×50 .
- **The 2,000 job backlog per robot exaggerates F11.** The real API caps a robot at 3 queued jobs, so in normal use the old claim query is much cheaper than in this benchmark.
- **Cloud noise.** Old runs vary a lot between repeats, up to 44 % for old, 1 API, 400 robots, pool 50 (144.9 to 209.2 jobs/s, Table 6), which is why medians and ranges are reported. New varies by under 3 % at 100 and 400 robots.

12. Conclusion

On the same Azure hardware, the SIT315 version of the dispatch path completes 3.44x more jobs per second than the SIT331 version on one API at 400 robots, and 6.05x with two APIs, cutting the time per job from 2,263 ms to 374 ms. At light load the gain (1.90x) comes from doing less database work per request. Under heavy load it comes from async I/O and the concurrency limiter, which keep latency flat where the old version degrades and then collapses (0 jobs per second at 5,000 robots). A second replica adds 1.76x for new but nothing at light load, and a bigger connection pool adds +16 to +22 % for new at 100 and 400 robots but nothing clear for old, because new's waiting had moved into the pool. The next limit is the shared database.

13. How to run

Build and push the images (from `Module_4/RobotControllerApi`):

```
git archive old | docker build -t robot-api:old -
git archive new | docker build -t robot-api:new -
docker build -t robot-loadgen -f loadgen/Dockerfile .
```

Run the tests (Docker must be running; the integration tests need a running API and are excluded):

```
dotnet test tests/RobotControllerApi.Tests/RobotControllerApi.Tests.csproj --filter
"FullyQualifiedName!~Integration"
```

On each API VM, start a version with a pool size: `~/api.sh old|new|stop [pool]`

On the load VM, the full grid and the stress test: `P00L=25 ./grid.sh` and `P00L=25 ./stress.sh`. Each runs the load generator against the load balancer:

```
docker run --rm -v $OUT:/loadgen/results robot-loadgen bench --api http://10.10.1.100 --db "  
<connection string>"  
  --hash-key <key> --label new-2-replica --robots 10,100,400 --backlog 2000 --duration 30 --warmup  
5 --repeats 3
```

Raw results are in `loadgen/results/` (`bench.csv`, `db_check.csv`, `run-info.txt`, the logs and the NBomber reports per run). Every table in this report is generated from those files by `Module_4/Report/make_tables.py`.

Acknowledgement of AI Use

I acknowledge that AI (Claude) has been used to review and edit this report. The final work is completely my own.